



Introduction

The [Rainbow](#) team asked us to review and audit their Swap Aggregator smart contracts. We looked at the code and now publish our results.

Scope

We audited commit [c696b5adca575903300e8af597e44c42a42b696f](#) of the [rainbow-me/swap-aggregator](#) repository. In scope were the following contracts:

- `routers/BaseAggregator.sol`
- `libraries/TransferHelper.sol`
- `RainbowRouter.sol`
- `interfaces/IERC20.sol`
- `interfaces/IDAI.sol`
- `interfaces/IERC2612.sol`
- `interfaces/IWETH.sol`

All other project files and directories, including tests, along with external dependencies and projects, game theory, and incentive design, were excluded from the scope of this audit. External code and contract dependencies were assumed to work as documented and back-end services provided by Rainbow were assumed to act in the best interest of the protocol.

Overall Health

In general, we found the project's codebase to be concise, readable, organized, and well commented. Together, all of this made the codebase easy to reason about. The project emphasizes gas efficiency; this can sometimes, directly or indirectly, lead to security concerns, which we raise issues about in the report. From the beginning of the audit, the Rainbow team expressed commitment to finding the right balance between gas costs and contract security for their users. Throughout the audit, the Rainbow team was highly available, responsive, and a pleasure to work with.

System Overview

The Swap Aggregator protocol is primarily meant to facilitate swaps via the front end that the Rainbow team controls. This leads to some design decisions that can seem rather curious outside of this context - such as allowing swap callers to specify their own transaction fees that will be paid to Rainbow.

The `RainbowRouter` merely retains any specified fees, forwards the swap request to the specified aggregator endpoint (assuming the aggregator has been whitelisted by the Rainbow team), and, as necessary, forwards the result of the swap back to the swap initiator.

Privileged Roles

The system has a single administrative role, and that is the `owner` of the `RainbowRouter`. The `owner` can transfer ownership, modify the whitelist for swap aggregator endpoints, and withdraw ETH and tokens owned by the `RainbowRouter` contract.

External dependencies and trust assumptions

For any liquidity aggregator endpoint that successfully facilitates a swap, the `RainbowRouter` grants the aggregator unlimited approval for the input token. Once this unlimited approval is granted, there is no mechanism for rescinding it. Even in the case where an aggregator is subsequently removed from the aggregator whitelist, any associated outstanding approvals will remain. (**Update:** *The Rainbow Swap Aggregator no longer grants unlimited approvals, please see the additional updates below.*)

Together with the fact that the whitelisted aggregators are responsible for interpreting and executing all swap requests, with little to no validation logic for the relayed swap call data or sanity checks of the swap results, the Swap Aggregator protocol and its users are explicitly dependent on the proper functioning of those aggregator endpoints.

By extension, given the fact that the protocol `owner` can add and remove aggregator endpoints which have the potential to execute arbitrary functions, protocol users must explicitly trust the protocol `owner` as well.

Update: The majority of the following issues have either been fixed or partially fixed by the Rainbow team. Our analysis of those mitigations is limited to the specific changes made to cover the issues and disregards all other unrelated changes in the associated pull requests and in the codebase.

The fixes were extensive and were implemented across various independent branches, which could result in merge conflicts. Given the number of changes, the project may benefit from a final refactor to make the codebase more cohesive and a final audit to ensure that no regressions or new vulnerabilities were introduced during the final merge and refactoring steps.

Additional Update: After our initial audit outlined throughout this report, the Rainbow team continued to iterate on their Swap Aggregator. They requested us to review and audit changes stemming from those iterations prior to putting the system into production. Listed below are the PRs reviewed and a brief description of what was addressed by each.

- [PR #63](#), a more gas-efficient ERC20 token transfer library was used in favor of Uniswap's token transfer library
- [PR #69](#), swap data validator logic was removed in favor of explicit allowances
- [PR #72](#) and [PR #74](#), cleaned up the removed validator logic and were continuations of the efforts of PR #69
- [PR #73](#), increased the precision of the fees their contract will collect
- [PR #76](#), further streamlined their swap logic to enforce complete allowance usage during swaps

In this batch of PRs we found only minor issues, and the Rainbow team addressed any of our concerns in real-time. At the end of our review, we found the codebase to be substantially improved in comparison to the initial audit. The Rainbow team did an excellent job of reducing complexity, gas usage, and the attack surface of their Swap Aggregator.

Findings

Here we present our findings.

Critical severity

[C01] Full token balance of arbitrary token transferred to user

The `BaseAggregator` contract is responsible for the bulk of the logic around facilitating swaps and collecting fees from those swaps for the protocol.

The contract's `fillQuoteEthToToken` function accepts ETH to swap for an ERC20 token at a specified `buyTokenAddress`. If the swap is successful, the function then transfers [the contract's entire balance of the token at the specified `buyTokenAddress`](#) to the swap initiator.

However, there are other functions in the contract, namely `fillQuoteTokenToToken`, that collect protocol fees as tokens that are then held on this contract. Taken together, the current design of `fillQuoteEthToToken` allows anyone to withdraw any fees that were collected as ERC20s and that are still held on the contract. Even a non-malicious actor swapping just a small amount of ETH for a token that the protocol has a balance of will receive that *entire* balance.

To avoid the potential for anyone to withdraw fees collected as ERC20s, consider adding logic to restrict the outgoing transfers of the contract via `fillQuoteEthToToken` only to funds received from the swap. Alternatively, consider immediately sending collected fees to a fee collection address that is distinct from the main protocol.

Update: Fixed in [pull request #36](#).

[C02] Lack of validation can lead to stolen assets

The `fillQuoteTokenToToken` function facilitates swaps which begin and end with ERC20 tokens. The function intends users to provide a `sellTokenAddress` and a `sellAmount` which should be the same as those used in the `swapCallData`.

However, there is no validation between `sellTokenAddress` and `swapCallData` nor `sellAmount` and `swapCallData`. This, paired with infinite approvals to aggregators, could allow bad actors to specify swaps which sell tokens other than the `sellTokenAddress` or more tokens than provided by `sellAmount`.

In such cases, the aggregator would swap the funds specified in the `swapCallData` and [send back any tokens received](#) from the ERC20 token contract specified by the `buyTokenAddress`.

Similarly, the `fillQuoteTokenToEth` function has the same vulnerability, the only difference being that the assets returned from the swap are ETH rather than ERC20 tokens.

Any ERC20 tokens held on the contract are susceptible to being stolen with this method.

Both internal functions have external functions that pass through the arguments, namely `fillQuoteTokenToToken` and `fillQuoteTokenToEth`, respectively. External function visibility coupled with the fact that funds are collected on this contract and can be collected in the form of ERC20s, makes it reasonable to expect that assets will be at risk from these vulnerabilities.

In order to mitigate this attack vector, consider avoiding infinite approvals, not holding funds on this contract, or implementing additional logic to ensure that malicious actors cannot swap tokens they do not provide or swap a token amount greater than they provide.

Update: Fixed in [pull request #37](#).

High severity

None.

Medium severity

[M01] Locked funds

In the `BaseAggregator` contract, both the `fillQuoteTokenToToken` and `fillQuoteTokenToEth` functions swap either from tokens to other tokens or from tokens to ETH, respectively.

First, both functions [move funds from the user to the contract](#) and then subsequently [call an aggregator with data for a swap](#). There is no authentication to ensure that the swap data provided actually swaps all of the tokens intended to be sold. As a result, in the case of left-over tokens, the contract will simply keep them instead of sending them back, leading to funds being unintentionally held by the contract.

In both functions, it is also possible to pass along ETH to the aggregators to facilitate the swap. This is required by some integrations, such as 0x, where ETH can be required to satisfy aggregator swapping fees. However, if provided on a swap that does not actually require ETH fees, the transferred ETH could get locked in the aggregator contract.

Similarly, both `fillQuoteTokenToToken` and `fillQuoteEthToToken` functions finish swaps by [transferring tokens at the `buyTokenAddress` back to the user](#). If a user were to input an incorrect `buyTokenAddress`, the actual tokens that were the result of their swap would become locked in the contract.

And, lastly, in the `fillQuoteEthToToken` function, if the [swap data that is executed](#) does not use the ETH sent with the message, it is possible for that ETH to be locked in the aggregator contract.

In order to avoid locking user funds in either the `RainbowRouter` contract or in the integrated aggregator contracts, consider restricting when ETH can be sent to the contract. Also consider implementing logic to ensure that all funds sent to the protocol were either used or returned to the user.

Update: Partially fixed in [pull request #40](#) and [pull request #37](#). All recommended checks, aside from the checks around sending ETH, were implemented.

Low severity

[L01] Lack of indexed parameters in events

Throughout the `RainbowRouter` contract, there are `address` parameters in events that are not indexed.

Consider [indexing event parameters](#) to avoid hindering the task of off-chain services searching and filtering for specific events.

Update: Fixed in [pull request #41](#).

[L02] Superfluous functions in the ERC2612 interface

The [IERC2612.sol](#) file provides an interface with the `permit` function signature for standard ERC2612 style permit calls. It is used in the [TransferHelper](#) library for calling `permit` on arbitrary token contracts.

The interface also provides function signatures for the `_nonces` and `version` functions, despite the fact that those are not included in the [official EIP](#). Adding extraneous function signatures to the interface can lead to confusion and potentially error-prone integrations.

In order to keep the code clean, clear, and concise, consider removing the `_nonces` and `version` function signatures from [IERC2612.sol](#). If additional function signatures are required for testing, consider defining them in a separate file.

Update: Fixed in [pull request #42](#).

[L03] Lack of input validation

In the [RainbowRouter](#) contract, the [withdrawEthFees](#) and [withdrawTokenFees](#) functions do not validate their `to` address parameter, which determines where funds are sent. This makes it possible for the contract `owner` to inadvertently pass in the zero address and, as a result, potentially lose access to withdrawn assets.

Additionally, in the same contract, the [transferOwnership](#) function fails to validate that the value of the `newOwner` parameter is different than the existing value of `owner`. Even when those values are identical, an [OwnerChanged](#) event is emitted, which, aside from wasting gas, could potentially lead to confusion for anyone subscribed to such events.

To avoid the potential for erroneous values to result in unexpected behaviors or wasted gas, consider adding input validation for all user-controlled input, including owner-only functions.

Update: Fixed in [pull request #43](#).

[L04] Misleading inline documentation

We found the following inline documentation to be misleading and believe that it could benefit from further clarification and/or additional specificity:

- In the `BaseAggregator` contract, the comment on [line 124](#): `"the swapCallData is passing sellAmount - feeAmount as the input"`, is misleading. In fact, nothing about `swapCallData`, including what it is passing in as the "input", is guaranteed by the logic of the smart contract.

Clear inline documentation is fundamental to outlining the intentions of the code. Mismatches between the documentation and the implementation can lead to serious misconceptions about how the system is expected to behave. Therefore, consider fixing any discrepancies to avoid confusion for developers, users, and auditors.

Update: Fixed in [pull request #44](#).

[L05] Naming issues

To favor explicitness and readability, parts of the contracts may benefit from better naming.

Our suggestions are:

- The `withdrawEthFees` function and `EthFeesWithdrawn` event should be named `withdrawEth` and `EthWithdrawn`, respectively, because there is no logic that restricts ETH withdrawals to only the collected fees.
- Similarly, the `withdrawTokenFees` function and the `TokenFeesWithdrawn` event should be named `withdrawToken` and `TokenWithdrawn`, respectively, because there is no logic that restricts token withdrawals to only the collected fees.
- The `permitSignature` argument, used in the `TransferHelper` library and `BaseAggregator` contract, should be named `permitData`. The actual signature values, `v`, `r`, and `s`, comprise only a small part of that argument.

Consider renaming any potentially confusing or misleading parts of the codebase to increase overall code clarity.

Update: Fixed in [pull request #45](#).

[L06] Outdated Solidity version in use

An outdated Solidity version, [0.8.7](#), is currently in use. As Solidity is now under a fast release cycle, consider using the latest compiler version at the time of deployment ([presently 0.8.11](#)) to incorporate the latest features and bug fixes. Note that although Solidity uses semantic versioning, it may introduce breaking changes between its minor versions, which should be taken into consideration.

Update: Fixed in [pull request #46](#).

[L07] Some public functions should be external visibility

In Solidity, functions with `public` visibility can be used both internally and externally. In the case a function is only ever used externally, it is best practice to limit the visibility to `external` rather than `public`. Below is a list of functions that have `public` visibility that can be limited to `external` visibility.

- `transferOwnership`
- `fillQuoteEthToToken`
- `fillQuoteTokenToToken`
- `fillQuoteTokenToTokenWithPermit`
- `fillQuoteTokenToEth`
- `fillQuoteTokenToEthWithPermit`

To improve the explicitness of the codebase, and to reduce gas usage, consider changing the visibility of these functions to `external`.

Update: Fixed in [pull request #47](#).

[L08] Unprotected receive function

The `receive` function in the `RainbowRouter` contract is devoid of any logic that restricts when or what address can send ETH to the contract.

Although this `receive` function is required for some of the aggregator integrations, such as 0x, there is no reason that it needs to be completely open. In fact, the open design could have the undesirable side effect of making it easier for ETH to be mistakenly sent to the `RainbowRouter` contract.

To reduce the propensity for ETH to accidentally be deposited into the contract, consider restricting the `receive` function to only accept funds during a swap or from whitelisted addresses.

Update: Fixed in [pull request #48](#).

Notes & Additional Information

[N01] TransferHelper may discard error messages

Throughout the `TransferHelper` library and the `BaseAggregator` contract, low-level `calls` are followed by `require` statements that evaluate the success of those low-level calls. If the calls are

unsuccessful, then the `require` statements emit their own error messages, disregarding any error messages passed back from the calls.

In favor of more helpful error messages for users and while debugging, consider bubbling up error messages from low level calls. If implementing such functionality, consider employing community vetted code, such as OpenZeppelin's `Address` library.

Update: Fixed in [pull request #37](#) and [pull request #51](#). The former PR makes it so that low level calls in `BaseAggregator` now bubble up revert strings. The latter PR replaces the custom `TransferHelper` library and imports a community vetted dependency instead.

[N02] Coding style deviates from Solidity Style Guide

The codebase deviates from the recommended [Solidity Style Guide](#).

Most notably, the function order throughout the codebase does not always follow the [recommended order of: constructor, receive, fallback, external, public, internal, private](#).

To increase overall code readability, consider reordering the functions throughout the codebase and conforming to the Solidity Style Guide where possible.

Update: Fixed in [pull request #49](#).

[N03] Fees are set by the user

Throughout the `BaseAggregator` contract, and, in turn, the entire protocol, swap fees are set by the swap initiator. That is to say, the caller of the swap functions specifies the fees for their swap transactions.

This pattern provides users a trivial way to circumvent paying any fees on their swaps, simply by specifying a `0` fee.

To ensure fees are always collected from swaps, consider using a fee set in the contract's storage, or rejecting user-specified fees less than a minimum fee, to prohibit users from setting `0` fees. Alternatively, to avoid confusion, consider bolstering the documentation around this design decision.

Update: Not fixed.

The Rainbow team states:

We only care about fees when users interact with the smart contract from any of our frontends (our mobile apps and maybe the web in the future).

[N04] DAI style permits restricted to DAI contract

The `permit` function in `TransferHelper.sol` has the ability to call the `permit` function on both `DAI` and `ERC2612` implementations. Currently, the way the function determines which permit style to use is by considering which address the call will be made to. It uses `DAI` style permits if the contract matches the `DAI` address and uses the `ERC2612` standard otherwise.

This restricts the ability to call `DAI` style permits on contracts other than `DAI`. More flexibility may be desirable. Consider adopting the more common approach of determining the style of the `permit` implementation to use based on the length of the permit data passed in. Alternatively, in favor of gas savings, consider adding a boolean flag to select which style is intended to be called to ensure the function can handle both `DAI` and `ERC2612` style permits regardless of the contract address.

Update: Fixed in [pull request #50](#).

[N05] Low level contract calls do not check for code

Several low-level calls in the `TransferHelper` library and `BaseRouter` contract expect that the addresses calls are made to are, in fact, contracts. However, they do not ensure that there is code at those addresses. This can lead to low-level calls to Externally Owned Accounts (EOAs) that return success but that actually do not carry out the expected actions.

Consider adding a check for code before these low-level calls to ensure EOAs are not being called when the calls are expected to be made to contracts. Alternatively, consider using OpenZeppelin's `Address` library to facilitate these low-level calls, as it will check for the existence of a contract before making such calls.

Update: Not fixed.

The Rainbow team states:

| All the external calls are made to pre-approved addresses set by ourselves.

[N06] Rewriting standardized code

Throughout the codebase, code for which there are open source, community-vetted implementations has been rewritten rather than imported from those public codebases. For example:

- `BaseAggregator.sol` reimplements OpenZeppelin's `ReentrancyGuard.sol`
- `IERC20.sol` reimplements OpenZeppelin's `IERC20.sol`
- `IDAI.sol` reimplements Uniswap's `IERC20PermitAllowed.sol`
- `IERC2616.sol` reimplements Uniswap's `IERC721Permit.sol`

- `TransferHelper.sol` reimplements Uniswap's `TransferHelper.sol`

Rewriting highly vetted and public dependencies introduces the likelihood of introducing errors. In order to avoid implementation mistakes, consider importing standardized code in favor of rewriting it.

Update: Partially fixed in [pull request #51](#). Changes address everything aside from the reimplementing of `ReentrancyGuard` in `BaseAggregator`.

[N07] Reaffirm SPDX license identifier

Throughout the codebase, the `SPDX license identifier` is specified as `Unlicense`. As this license is not used for other software that the Rainbow team has released, and given that this identifier differs from the special value `UNLICENSED` by only a single character, consider reaffirming the license the smart contracts will be released under. Additionally, to increase overall clarity, consider adding additional references to the chosen license in the project's `README` and supporting documentation.

Update: Fixed in [pull request #52](#), where the `SPDX license identifiers` are updated, and [pull request #51](#), where the remaining files referencing the `Unlicense` are removed.

[N08] Typos

The codebase contains the following typos:

- `Substract` should be `Subtract`
- `fund` should be `funds`
- `allowlist` should be `swapTargets`
- `solver` should be `aggregator`

Consider correcting these typos to improve code readability.

Update: Fixed in [pull request #53](#).

[N09] Inefficient logic flow

The fee calculation and collection in the `BaseAggregator` contract's `_fillQuoteTokenToEth` function will calculate the fee and attempt to send the amount reduced by the fee to the user. Immediately after, it checks if there was any ETH returned from the swap in the first place.

With the order of the current logic, it is possible for the contract to calculate a fee on an empty ETH return and make a 0 ETH transfer to the user specifically when 0 ETH is returned but a fee was specified.

In order to avoid unnecessary 0 ETH transfers, consider first checking that the amount of ETH returned is non-zero, and then calculate and send the proper amount of ETH.

Update: Fixed in [pull request #54](#).

[N10] Undocumented implicit approval requirements

In the `BaseAggregator` contract, the `_fillQuoteTokenToEth` and `_fillQuoteTokenToToken` functions implicitly assume that the necessary approvals have been granted to transfer tokens from the `msg.sender` to the `BaseAggregator` as the initial step in a swap.

In favor of explicitness and to improve the overall clarity of the codebase, consider documenting all approval requirements in the relevant functions' inline documentation.

Update: Fixed in [pull request #55](#).

[N11] Unused interface

The `IWETH` interface defined in `IWETH.sol` is not used within the codebase.

Consider removing the interface from the codebase. Alternatively, if it is intended to be used, consider either implementing it or documenting the reason for keeping it.

Update: Fixed in [pull request #56](#).

Conclusions

2 critical severity issues were found. Some changes were proposed to follow best practices and reduce the potential attack surface.